

Case Study

Vantar Communications — Network Operations Copilot & Multi-Agent Customer Care Orchestration

Confidential Internal Engagement Documentation

Engagement Period: March 2025 – January 2026

- [1. Executive Summary](#)
- [2. Client Background](#)
- [3. Business Problem](#)
- [4. Constraints](#)
- [5. Discovery Transcript](#)
 - [5.1 Kickoff — Week 1](#)
 - [5.2 NOC Shadowing — Week 2](#)
 - [5.3 Customer Care Shadowing — Week 3](#)
 - [5.4 Capability Mapping and ROI — Week 4](#)
- [6. Architecture Workshops](#)
 - [6.1 Business & Information Architecture](#)
 - [6.2 AI/Platform Architecture — Whiteboard Session, Week 9](#)
 - [6.3 Security & Identity Architecture](#)
 - [6.4 Integration & API Strategy](#)
- [7. Technical Debates](#)
 - [7.1 Model Selection — Cost-Tiered Routing](#)
 - [7.2 Semantic Caching for High-Repeat Query Patterns](#)
 - [7.3 Agent-to-Agent Handoff Design — the Debate That Preceded the Incident](#)
- [8. Executive Reviews](#)
 - [8.1 Architecture Review Board — Week 20](#)
 - [8.2 CTO Cost Governance Review — Week 26](#)
 - [8.3 Post-Incident Executive Review — Week 40 \(see Section 14\)](#)
- [9. Final Architecture](#)
- [10. Delivery Roadmap](#)
- [11. Risks](#)
- [12. Governance Model](#)
- [13. Production Rollout](#)
- [14. Production Incident — Month 9](#)
 - [14.1 Incident Summary](#)
 - [14.2 Incident Response Transcript](#)
 - [14.3 Remediation](#)
- [15. Lessons Learned](#)
- [16. Enterprise Architecture Artifacts](#)
- [17. Architecture Decision Records \(ADRs\)](#)
- [18. AI Evaluation Strategy](#)
- [19. Operational Runbook](#)
- [20. Future Roadmap](#)

1. Executive Summary

Vantar Communications, a national telecommunications carrier serving approximately 28 million subscribers across mobile, broadband, and enterprise services, engaged an Enterprise AI Architecture team to modernize two operationally critical but organizationally separate functions: Network Operations Center (NOC) incident triage, and multi-channel customer care agent orchestration.

The eleven-month engagement delivered a NOC copilot that correlated alarms across Vantar's event-streaming backbone to accelerate root-cause identification, and a LangGraph-based multi-agent customer care system coordinating specialized agents (billing, technical support, retention, provisioning) behind a unified conversational interface across chat, voice, and app channels. Both systems shared a common LLM gateway and cost-governance layer — a deliberate architectural choice given Vantar's scale, where uncontrolled token spend was identified in discovery as a real financial risk, not a hypothetical one.

That risk materialized in Month 9: during a major regional network outage, a cascading agent-to-agent interaction loop between the customer care system's technical-support and retention agents — triggered by an edge case in how the two agents handed off a specific category of outage-related customer inquiry — caused a sustained spike in LLM API calls that went undetected for approximately five hours before cost-governance alerting caught it, resulting in an estimated \$340,000 in unplanned inference spend during a single incident window. This became the defining case study for Vantar's subsequent investment in agent-to-agent loop detection and hard circuit-breaker controls.

Key outcomes: - Mean time to root-cause identification for Sev-2 and above network incidents reduced from approximately 38 minutes to 11 minutes - Customer care first-contact resolution rate improved by 19 percentage points on in-scope inquiry categories - One material cost-governance incident (\$340K unplanned spend in a single event) triggering mandatory circuit-breaker architecture across all multi-agent workflows - LLM inference cost per resolved customer interaction reduced by 46% post-remediation through combined circuit-breaker and semantic-caching improvements

2. Client Background

Vantar Communications operates a nationwide fiber and wireless network with a NOC staffed 24/7 across three regional operations centers, and a customer care organization handling roughly 40 million contacts annually across phone, chat, and self-service app channels — split between an internal care organization and two outsourced BPO partners handling overflow volume.

Vantar's Chief Technology Officer, Elena Kowalski, and SVP of Customer Experience, Marcus Reid, co-sponsored the engagement, reflecting its cross-functional scope. Vantar's existing technology stack included a substantial Kafka-based event-streaming backbone (already mature, predating this engagement, used for network telemetry), a legacy IVR system nearing end-of-vendor-support, and three separate CRM-adjacent systems accumulated through prior acquisitions that customer care agents had to toggle between manually.

3. Business Problem

NOC incident triage. Vantar's network generated an enormous alarm volume — on a bad day, tens of thousands of individual alarm events, the large majority correlated symptoms of a smaller number of actual root-cause incidents. NOC engineers manually correlated alarms using tribal knowledge and static runbooks, a process discovery found took an average of 38 minutes to reach root-cause identification for significant incidents, directly extending customer-facing outage duration.

Customer care fragmentation. A customer contacting Vantar about a billing dispute that turned out to be related to a service outage might be bounced between separate billing and technical-support queues, each with limited visibility into the other's context, requiring the customer to re-explain their issue multiple times — a top driver of customer satisfaction complaints per Reid's team's own data, and a meaningful contributor to Vantar's above-industry-average customer churn rate.

4. Constraints

1. **Scale.** Vantar's contact volume (40 million customer interactions annually) and network telemetry volume meant that architectural decisions with even small per-transaction cost or latency implications had large aggregate consequences — a design principle the team had to hold consistently, not just acknowledge abstractly.
 2. **Regulatory (telecom-specific).** Customer communications related to service outages and billing were subject to state public utility commission requirements in several of Vantar's service territories, requiring accurate, auditable records of what customers were told and when, particularly during declared outage events.
 3. **BPO integration.** Roughly 35% of customer care volume was handled by two outsourced BPO partners using their own agent-desktop tooling, not Vantar's internal systems — any customer care AI capability needed a path to extend to BPO agents, not just Vantar's internal care organization, or a large share of interactions would be excluded from the improvement.
 4. **Legacy IVR sunset timeline.** The existing IVR vendor had announced end-of-support within 18 months, creating schedule pressure to have a viable voice-channel replacement path, though full IVR replacement was ultimately scoped as a separate, parallel initiative rather than folded into this engagement.
 5. **Cost governance.** Given Vantar's scale, LLM inference cost per interaction was treated as a first-class architectural concern from day one, not an operational afterthought — CTO Kowalski was explicit in kickoff that an architecturally elegant but cost-uncontrolled design would not be approved regardless of its other merits.
 6. **Data privacy across acquired entities.** The three legacy CRM systems from prior acquisitions had different historical data-handling practices; a unified customer-context retrieval layer had to reconcile these without creating new privacy-policy violations for data collected under each acquired entity's original terms.
-

5. Discovery Transcript

5.1 Kickoff — Week 1

Present: Elena Kowalski (CTO), Marcus Reid (SVP Customer Experience), Head of Network Operations (Priya Ramanathan), the Enterprise AI Architect (EAA).

Kowalski: I want to set an expectation early. We are a high-volume, thin-margin business. I have seen AI pilots at other carriers produce genuinely impressive demos that turned out to be financially unviable at our actual call volume once someone did the unit-economics math. I don't want architecture reviews that only talk about capability — I want cost per transaction in every conversation from day one.

EAA: That's exactly the right instinct at your scale, and I'd rather build that discipline into the architecture from the start than retrofit it after a cost surprise. I'd propose every agent or workflow we design carries an explicit cost budget per interaction as a design constraint alongside its functional requirements — not a metric we check after the fact, but something that shapes model selection, caching strategy, and orchestration design from the first whiteboard session.

Reid: From my side, the thing that's killing our customer satisfaction scores is fragmentation — a customer has to re-explain their problem three times because billing, tech support, and retention are organizationally and technically separate. If AI just makes each of those three siloed experiences individually slightly better, that's not solving the actual problem my team is dealing with.

EAA: That reframes the design goal usefully — the value isn't "smarter billing bot" or "smarter tech support bot" in isolation, it's *continuity of context across the customer's full interaction*, regardless of which specialized capability is actually handling a given moment of the conversation. That points toward a multi-agent architecture with a shared context layer and an orchestration mechanism that can hand off between specialized agents without the customer perceiving a "restart" — which is architecturally more complex than a single bot, and I want to be upfront that complexity has both cost and reliability implications we'll need to manage deliberately, not something we get for free.

Ramanathan: On the network side, my engineers already know how to correlate alarms — the problem isn't lack of expertise, it's that at 2am during a major event, a junior engineer without ten years of pattern-matching experience is working from static runbooks that don't capture what senior engineers actually do intuitively.

EAA: That's a useful distinction from the customer care problem — this isn't primarily an information-fragmentation problem, it's a **tacit-knowledge-capture and pattern-correlation** problem. That points toward a different kind of system: one that learns correlation patterns from historical incident data and senior engineers' resolution paths, surfaced as a triage copilot for the engineer actually on shift, not a fully autonomous incident-resolution system — network changes remain a human decision given the safety and service-continuity stakes.

5.2 NOC Shadowing — Week 2

Senior NOC Engineer (Tomasz Wojcik), during a live minor incident: See this alarm storm? [pointing at a dashboard flooding with alerts] Ninety percent of these are downstream symptoms of one fiber cut upstream. I know that because I've seen this exact alarm signature pattern probably two hundred times in eleven years here. A newer engineer would chase each alarm individually before realizing they're all one thing.

EAA: Is that pattern documented anywhere, or is it purely experience-based?

Wojcik: Purely experience. We have runbooks for known failure categories, but this specific correlation pattern — which alarms cluster together for which root causes — isn't written down anywhere. It's the kind of thing that walks out the door when senior engineers retire.

This session directly shaped the decision to build the NOC copilot around **learned alarm-correlation patterns mined from historical incident-resolution data**, explicitly capturing the kind of tacit knowledge Wojcik described, rather than a purely rules-based runbook-automation approach — a genuine knowledge-capture and retention benefit beyond the immediate triage-speed goal.

5.3 Customer Care Shadowing — Week 3

Customer Care Agent (Denise Alvarez), listening to a call queue in real time: This customer's been

transferred twice already. Started in billing because their bill spiked, but the spike is because tech support extended their service credit incorrectly last month after an outage, and now I have to pull up three different systems to piece together what actually happened before I can even start solving today’s problem.

EAA: When you finally do piece it together, is that reconstructed context saved anywhere for the next agent, if this isn’t resolved today?

Alvarez: Not really — case notes exist, but they’re inconsistent in quality and the next agent still has to read through everything rather than getting a synthesized picture.

This directly shaped the design of a **shared conversational context and case-synthesis layer** — not just agent handoff, but active synthesis of the customer’s cross-channel, cross-issue history into a coherent working picture any specialized agent (human or AI) could pick up from, described further in Section 6.1.

5.4 Capability Mapping and ROI — Week 4

Capability	Current State	AI Opportunity	Impact	Feasibility
NOC alarm correlation / root-cause triage	Manual, experience-dependent	Learned-correlation triage copilot	Very High	Medium
Customer care context continuity	Fragmented across systems/channels	Shared context + multi-agent orchestration	Very High	Medium-High
Autonomous network remediation	None	Explicitly out of scope — remains human decision	N/A	N/A
IVR replacement	Legacy, end-of-support	Deferred — separate parallel initiative	High	Low (scoping)
Predictive churn intervention	Existing separate model	Integration point for retention agent, not rebuild	Medium	High

EAA: I want to flag the IVR replacement question directly, since it came up given the vendor’s end-of-support timeline. I’d recommend against folding full IVR replacement into this engagement’s scope — it’s a large, distinct undertaking with its own vendor selection and telephony-integration complexity, and bolting it onto this engagement risks both scope creep and diluting focus on the two problems we’ve actually done discovery against. I’d recommend a parallel, separately-scoped initiative, with this engagement’s voice-channel agent orchestration designed to be IVR-vendor-agnostic so it can integrate with whatever replacement is eventually selected.

Kowalski: Agreed — keep this engagement focused. I’ll stand up a separate initiative for IVR replacement.

6. Architecture Workshops

6.1 Business & Information Architecture

Enterprise Data Architect (Wei Zhang): How do we build the shared customer-context layer Reid’s team needs, given the three separate legacy CRM systems with different historical data practices?

EAA: I’d propose a **Customer Context Service** that doesn’t attempt to physically consolidate the three CRM systems — that’s a multi-year data-migration project on its own and not something this engagement should take on — but instead provides a real-time federated view, querying each system live and synthesizing a unified working context per interaction, respecting each source system’s original data-handling terms rather than creating a new consolidated store that might violate acquisition-specific privacy commitments.

Zhang: That’s more architecturally complex than a single consolidated database.

EAA: It is, and I think that complexity is justified here specifically because of the privacy-terms constraint Elena’s team flagged — a consolidated store would require re-consenting or re-papering data-handling terms across three different legacy customer bases, which is a legal and trust exercise well beyond this engagement’s scope and timeline. Federation lets us deliver the continuity-of-context value Marcus needs without taking on that separate, much larger undertaking.

6.2 AI/Platform Architecture — Whiteboard Session, Week 9

Platform Architect (Sam Ito): Walk through the customer care multi-agent design.

EAA: [at whiteboard] Customer interaction enters through any channel — chat, voice (via existing IVR for now), or app. An Orchestrator Agent, built on LangGraph, receives the interaction along with the synthesized context from the Customer Context Service, and routes to one or more specialized agents — Billing, Technical Support, Retention, Provisioning — based on the interaction’s actual content, not a rigid upfront menu-selection the way the current IVR forces customers into.

Ito: How do specialized agents hand off without the customer perceiving a restart, which was Marcus’s core complaint?

EAA: The Orchestrator maintains the canonical conversation state and context synthesis throughout the interaction — when it routes to, say, the Technical Support agent after starting with Billing, the Technical Support agent receives the full synthesized context, not a fresh conversation. From the customer’s perspective, they’re talking to one continuous entity; the specialization happens beneath that surface. This is architecturally similar in spirit to the aviation engagement’s separation of specialized agents behind a single orchestrator, though the consequence profile here is customer experience and cost, not safety, which changes how much autonomy we’re comfortable giving the orchestrator’s routing decisions.

Ramanathan (also present, having engaged with the NOC design): For the NOC copilot, is it also multi-agent, or a single system?

EAA: Different design, deliberately — the NOC copilot is closer to a single, focused reasoning agent with strong tool access (querying the event-streaming backbone, historical incident database, and the learned correlation model) rather than a multi-agent orchestration. The customer care problem genuinely benefits from specialized agents because billing, technical support, and retention require different tool access, different escalation paths, and arguably different “voice” in how they communicate — the NOC triage problem is more naturally a single expert reasoning over one integrated data view. I’d resist multi-agent architecture as a default pattern applied everywhere; use it where the problem’s structure actually calls for it.

6.3 Security & Identity Architecture

- OAuth 2.0/OIDC identity federation extending to BPO partner agent-desktop integrations (Section 4 constraint), with a distinct, narrowly-scoped identity class for BPO-originated sessions versus internal Vantar care agents
- Customer Context Service enforces per-source-system data-handling terms at the federation layer, not just in policy documentation — queries against acquired-entity legacy data automatically apply that entity’s original consent scope
- API gateway (Kong) fronting all agent and Customer Context Service endpoints, with rate limiting and cost-

attribution tagging per interaction, per channel, and per BPO partner — necessary given the cost-governance mandate from Week 1

6.4 Integration & API Strategy

- MCP server exposing Customer Context Service, billing system, provisioning system, and the churn-prediction model (existing, integration point not rebuild per Section 5.4) as tools consumed by the relevant specialized agents
 - Event-driven architecture (existing Kafka backbone, extended) for NOC alarm ingestion and correlation-model input
 - LLM Gateway: shared across both NOC copilot and customer care orchestration, providing unified cost tracking, model routing, and — critically, given the Month 9 incident — the architectural seam where circuit-breaker controls were ultimately implemented (Section 14.3)
-

7. Technical Debates

7.1 Model Selection — Cost-Tiered Routing

ML Engineer (Devon Park): Given the cost-per-interaction mandate, are we using a single model across all agents, or varying by task?

EAA: Varying by task, deliberately, and I'd frame this as the single biggest lever we have for hitting Elena's cost target without sacrificing quality where it matters. Simple, high-confidence classification tasks — initial intent routing, straightforward billing lookups — can run on a smaller, cheaper, faster model. Complex reasoning tasks — a Retention agent negotiating a save offer within policy constraints, or the NOC copilot correlating an ambiguous alarm pattern — justify a larger model's cost given the higher stakes and lower volume of those specific interactions. I'd propose a cost-tiered routing layer in the LLM Gateway that makes this an explicit, tunable policy, not a one-time model choice baked into each agent's code.

Park: How do we decide the tier boundary for a given task without just guessing?

EAA: Empirically, via the evaluation strategy — we measure quality degradation, if any, from routing a given task class to a cheaper model against a quality-holdout set, and only adopt the cheaper tier where degradation is within an agreed tolerance. I don't want cost-tiering decisions made by intuition about which tasks "feel" simple; the eval data should drive it.

7.2 Semantic Caching for High-Repeat Query Patterns

Ito: NOC alarms during a major event tend to be highly repetitive in pattern — could semantic caching meaningfully cut costs there?

EAA: Yes, and I'd extend that same principle to customer care — during a regional outage, Reid's team's own data shows an enormous spike in near-identical customer inquiries ("is there an outage in my area," "when will service be restored"). I'd propose semantic caching at the Orchestrator level for this specific pattern — recognizing semantically similar outage-status inquiries and serving a cached, recently-verified response rather than a fresh LLM generation for what is functionally the same question asked thousands of times during a major event. This is exactly the highest-value, highest-volume moment for caching to matter, similar to a pattern I've seen work well in an aviation-sector disruption-briefing context.

Park: What's the risk of a stale cached response during a fast-moving outage?

EAA: Real risk, and it needs an explicit freshness policy, not an open-ended cache. I'd propose a short TTL (a few minutes) tied to the outage-status data source's own update frequency, with cache invalidation triggered automatically whenever the underlying network-status system publishes an update — so the cache accelerates repeat queries between genuine status changes, not across them.

This became ADR-006, and the caching architecture proved directly relevant to the cost-governance remediation following the Month 9 incident (Section 14.3).

7.3 Agent-to-Agent Handoff Design — the Debate That Preceded the Incident

Reid (Week 18): For a customer whose billing issue is actually caused by an outage-related service credit, I want the Retention agent to be able to proactively suggest a goodwill credit if the Technical Support agent's diagnosis confirms an outage was the root cause — a fully automated handoff, not requiring the customer to separately contact retention.

EAA: I support the goal, and I want to flag a design risk in how we implement the automated handoff, given the systems involved will be reasoning somewhat independently before converging. If the Technical Support agent's outage-confirmation and the Retention agent's credit-eligibility check aren't both grounded in the exact same, single source of truth about the outage's status and scope, there's a real risk of the two agents' handoff loop re-triggering each other's re-evaluation logic if either agent's confidence in the correlation is borderline — essentially an oscillation risk in ambiguous cases. I'd recommend a hard, deterministic handoff contract: Technical Support either confirms outage-root-cause with a specific ticket reference or it doesn't, no partial-confidence handoff to Retention, and Retention's credit-eligibility logic treats that ticket reference as ground truth without re-deriving its own independent judgment about outage status.

Reid: That sounds like it could slow down legitimate cases where the correlation actually is a little ambiguous but clearly outage-related to a human.

EAA: That's a fair tension, and I don't think there's a free lunch here — I'd rather accept slightly reduced automation coverage on genuinely ambiguous cases, routing those to a human agent, than accept the oscillation risk I just described. I'll be honest that I can't fully rule out edge cases in a system this complex, which is exactly why I'd also want hard circuit-breaker limits on any agent-to-agent interaction loop regardless of how carefully we design the handoff contract — belt and suspenders, not relying on the contract design alone.

Reid: Let's ship the deterministic-handoff version, and I'll take the circuit-breaker recommendation to the ARB as a hardening item, not a launch blocker.

This decision — accepted with the circuit-breaker recommendation deferred rather than treated as a launch-blocking requirement — is directly relevant to the Month 9 incident and is revisited explicitly in the postmortem (Section 15).

8. Executive Reviews

8.1 Architecture Review Board — Week 20

Kowalski (chairing): I want a specific answer to the circuit-breaker recommendation from the Week 18 session. Is it in scope for launch or not?

EAA: My honest recommendation, restated directly for the board: I think it should be a launch requirement, not a post-launch hardening item, specifically because the failure mode we're guarding against — agent-to-agent interaction loops — has a cost-blast-radius that scales with your interaction volume, which per Elena's own Week 1 framing is exactly the kind of risk that looks small in a pilot and large at Vantar's actual scale.

Reid: I hear that, and I don't disagree with the risk in principle, but the deterministic-handoff-contract design we agreed in Week 18 should prevent the specific loop scenario we discussed. I'd rather launch on schedule with the contract design and treat circuit breakers as a fast-follow hardening item, given the schedule pressure from the IVR sunset timeline on the voice channel side.

Kowalski: I'm going to side with launching on schedule but I want the circuit-breaker work prioritized as the very next thing after launch, not a "someday" backlog item, and I want a defined maximum time-to-implementation, not an open-ended fast-follow.

EAA: I'll document my recommendation as a formal dissent in the ARB minutes, not to relitigate after the decision, but so there's a clear record for if this risk does materialize — which, transparently, is exactly what happened in Month 9.

This exchange, preserved in full, is directly referenced in the Month 9 postmortem (Section 15) as a case study in a correctly-identified risk that was consciously, defensibly deprioritized under real schedule pressure — and in what happened when it materialized.

8.2 CTO Cost Governance Review — Week 26

Kowalski: Give me the honest cost-per-interaction number against our target.

EAA: We're at \$0.34 per resolved customer care interaction against a target of \$0.30, and \$0.19 per NOC triage assist against a target of \$0.25 — so we're ahead of target on NOC, slightly behind on customer care, primarily driven by the Retention agent's negotiation-reasoning tasks, which are legitimately complex enough to need the larger model tier more often than we'd originally modeled.

Kowalski: Is \$0.04 over target something I need to worry about, or rounding error at your projected volume?

EAA: At 40 million annual interactions, \$0.04 per interaction is roughly \$1.6M annually — not rounding error, but also not something I'd recommend an emergency response to. I'd recommend continued semantic-caching expansion (Section 7.2) as the most promising lever to close that gap before considering a more aggressive cost-tier shift that might degrade Retention agent quality.

8.3 Post-Incident Executive Review — Week 40 (see Section 14)

Kowalski: I want to revisit the Week 20 decision directly. Was launching without circuit breakers the wrong call?

EAA: I want to answer this carefully rather than either defensively claim it was clearly right or simply say "I told you so," because I don't think either framing is accurate or useful. The decision was a genuine, informed tradeoff between two real considerations — schedule pressure with a defensible business reason, against a risk that was correctly identified and explicitly documented, including my dissent. It was defensible given what was known at the time. What I'd say plainly: the fast-follow prioritization commitment from that meeting should have had a harder deadline attached, and it didn't — five months elapsed between the Week 20 decision and the Month 9 incident, which is longer than "the very next thing after launch" should reasonably have taken.

Reid: I'll own that — the fast-follow commitment slipped due to competing priorities on my side, and I didn't escalate that slippage the way I should have.

Kowalski: That's the actual gap, then — not the original decision, but the lack of a hard tracked deadline on a documented, accepted risk. I want every ARB-documented risk-acceptance decision from now on to carry a mandatory remediation deadline with automatic escalation if it slips, not an open-ended "fast-follow."

9. Final Architecture

NOC Copilot - Single, tool-equipped reasoning agent (not multi-agent) with access to the event-streaming backbone, historical incident database, and a learned alarm-correlation model mined from senior engineers' historical resolution patterns - Human NOC engineer retains all remediation decision authority; the copilot's role is triage acceleration, not autonomous action - Semantic caching for repetitive alarm patterns during major events

Customer Care Multi-Agent Orchestration - LangGraph-based Orchestrator Agent maintaining canonical conversation state and synthesized cross-channel context - Specialized agents: Billing, Technical Support, Retention, Provisioning — each with distinct tool access and escalation paths - Deterministic, ticket-reference-based handoff contracts between agents (no partial-confidence handoffs), per Section 7.3 - Mandatory agent-to-agent circuit breakers (post-incident, Section 14.3): hard call-volume and loop-detection limits at the LLM Gateway layer, applied uniformly across all multi-agent workflows - Cost-tiered model routing, empirically validated per task class against quality-holdout evaluation - Semantic caching for high-repeat query patterns (notably outage-status inquiries during major events), TTL-bound and invalidated on underlying status-source updates - Customer Context Service: real-time federated view across three legacy CRM systems, respecting each source's original data-handling terms - BPO partner integration via scoped, distinct identity class

10. Delivery Roadmap

Phase	Duration	Scope
Discovery & Architecture	Months 1-2	Discovery across NOC and customer care domains
NOC Copilot Build	Months 2-5	Correlation model, triage copilot, pilot in one regional NOC
Customer Care Orchestrator Build	Months 3-7 (overlapped)	Orchestrator, specialized agents, Customer Context Service
ARB Review & Launch Decision	Month 6 (during build)	Circuit-breaker scope decision (Section 8.1)
Pilot Launch (Customer Care)	Months 7-8	Limited channel/region rollout
Full Rollout (Both Systems)	Months 8-9	Expansion to all regions, BPO integration
Cost Governance Incident & Remediation	Month 9	Agent-to-agent loop incident, circuit-breaker emergency build
Post-Remediation Stabilization	Months 9-11	Circuit breakers deployed platform-wide, cost optimization

11. Risks

Risk	Likelihood	Impact	Mitigation	Owner
Agent-to-agent interaction loops	Medium (realized — Section 14)	High (cost)	Circuit breakers, mandatory post-incident; deterministic handoff contracts	Platform Architecture
Cost-per-interaction overrun at scale	Medium	High	Cost-tiered model routing, semantic caching, continuous monitoring against target	CTO / Platform Architecture
Federated Customer Context Service latency/reliability across 3 legacy systems	Medium	Medium	Graceful degradation design; cached fallback context when a source system is slow/unavailable	Enterprise Data Architecture
BPO partner integration inconsistency	Medium	Medium	Scoped identity class, phased BPO rollout with dedicated support	Customer Experience
Regulatory record-keeping gaps for outage communications	Low	High	Auditable interaction logging meeting state PUC requirements	Legal / Compliance

12. Governance Model

- **Mandatory remediation deadlines on accepted risks:** post-incident governance addition (Section 8.3) — any ARB-documented risk-acceptance decision now carries a hard deadline with automatic escalation on slippage, replacing the open-ended “fast-follow” pattern implicated in the Month 9 incident
 - **Cost governance as a standing ARB agenda item:** every quarterly ARB review includes cost-per-interaction tracking against target, not just feature/capability review
 - **Circuit-breaker requirement:** now a standard, non-optional component of any multi-agent workflow design, platform-wide, not scoped per-project
 - **AI Change Advisory process:** any change to agent handoff contracts or orchestration routing logic requires Platform Architecture and Customer Experience joint sign-off
-

13. Production Rollout

Customer Care Orchestrator pilot launched in Month 7 across a limited set of regions and channels (chat only initially, voice integration following in Month 8). Full rollout including BPO partner integration completed by Month 9 — the same month the cost-governance incident occurred, during a period of genuinely elevated real-world load (a major regional network outage), which is precisely the high-stress condition under which the agent-to-agent loop risk from Section 7.3 materialized.

14. Production Incident — Month 9

14.1 Incident Summary

During a significant regional network outage affecting approximately 180,000 subscribers, customer contact volume spiked well above normal levels, with a large share of inquiries falling into the outage-related billing/credit category the Section 7.3 handoff design was built to handle. A specific edge case in the interaction between Technical Support and Retention agents — a customer inquiry pattern involving a partial, multi-service outage (only one of two bundled services affected) that fell into an ambiguity the deterministic ticket-reference handoff contract had not fully anticipated — caused the two agents to repeatedly re-invoke each other's evaluation logic for a subset of these specific dual-service cases, each re-invocation consuming LLM API calls without converging to a resolution or escalating to a human agent.

This loop pattern went undetected for approximately five hours — cost-governance alerting existed but was tuned to detect aggregate cost anomalies over a daily window, not the specific signature of a fast-accumulating loop within a narrow interaction category during an otherwise-expected volume spike (the overall spike in customer contacts during a major outage was itself expected and did not, on its own, look anomalous). The cumulative effect was an estimated \$340,000 in unplanned LLM inference spend concentrated in this narrow window before an engineer manually investigating an unrelated latency complaint noticed the pattern and manually disabled the affected handoff path.

14.2 Incident Response Transcript

Kowalski (emergency session, within 3 hours of detection): I want the exact mechanism, not a general description.

EAA: The dual-service partial-outage case wasn't explicitly covered by the deterministic ticket-reference contract from Section 7.3 — that contract assumed an outage confirmation applied unambiguously to the customer's full account, but a partial outage affecting only one of two bundled services created a scenario where Technical Support's confirmation was correct but incompletely scoped, and Retention's credit-eligibility logic, encountering that incomplete scoping, re-requested clarification from Technical Support rather than either proceeding or escalating to a human — and Technical Support's response to that re-request triggered the same ambiguity again, rather than breaking the loop.

Ito: Why didn't the aggregate cost monitoring catch this faster?

EAA: Because the monitoring was designed and tuned around the risk model we had at the time — aggregate daily spend anomalies — not a loop signature within a narrow interaction category during a period when overall elevated spend was itself expected and legitimate, given the outage was driving genuinely higher contact volume across the board. This is exactly the gap the circuit-breaker recommendation from Week 18 and Week 20 was meant to close, and it wasn't in place yet.

Reid: I want to say directly, for the record: this is the risk we accepted at Week 20 with an open-ended fast-follow commitment that I let slip. I'm not going to minimize that.

Kowalski: I appreciate that directness, and it matches my read of the situation. I want an immediate stopgap today, and I want the full circuit-breaker architecture built this month, not fast-followed again.

EAA: Immediate stopgap: I'd recommend a hard per-interaction-category call-count ceiling deployed today as an emergency change — any single customer interaction that triggers more than a small, fixed number of agent-to-agent handoff cycles automatically escalates to a human agent rather than continuing to loop, regardless of whether the underlying ambiguity is resolved. That's blunt and will occasionally escalate cases that could have resolved automatically with a few more cycles, but it caps the blast radius immediately while we build the more sophisticated circuit-breaker and loop-detection architecture properly.

Kowalski: Deploy that today. Full architecture, reviewed and tested, within three weeks — and I want weekly progress reporting to me directly until it ships.

14.3 Remediation

The emergency stopgap (hard per-interaction handoff-cycle ceiling) was deployed within 12 hours. The full circuit-breaker architecture, built over the following three weeks, added: real-time loop-signature detection at the LLM

Gateway layer (distinct from aggregate daily cost monitoring), per-interaction-category anomaly baselines that could distinguish “expected elevated volume during a declared outage” from “abnormal per-interaction call patterns,” and an explicit resolution to the dual-service partial-outage ambiguity in the handoff contract itself, closing the specific gap that triggered the incident.

15. Lessons Learned

1. **A correctly identified, explicitly documented, and consciously accepted risk still needs a hard enforcement mechanism to ensure its accepted mitigation actually happens on schedule.** The Week 20 ARB decision was a defensible tradeoff at the time it was made; the actual governance gap was the absence of a binding deadline on the “fast-follow” circuit-breaker commitment, which allowed competing priorities to silently push it past the point where it mattered. This directly produced the post-incident governance change requiring mandatory deadlines on all risk-acceptance decisions.
 2. **Monitoring tuned to yesterday’s risk model can miss today’s failure mode, especially during exactly the high-stress conditions where failures are most likely and most costly.** The aggregate daily-cost monitoring was reasonable when designed but wasn’t the right instrument for detecting a narrow, fast-accumulating loop during a period of legitimately elevated overall volume. Any future cost-governance design should explicitly ask “what would this monitoring miss during a high-stress event, not just during normal operations” as a design review question, not an afterthought.
 3. **Edge cases in agent handoff contracts are not fully eliminable by careful upfront design, which is exactly why circuit breakers are a necessary backstop, not a redundant one.** The Section 7.3 design was thoughtful and the dual-service ambiguity was a genuinely subtle edge case, not an obvious oversight. The postmortem’s core recommendation was explicit: any multi-agent system with the potential for agent-to-agent re-invocation loops needs hard, deterministic circuit breakers as a non-negotiable architectural component, independent of how well-designed the handoff logic is believed to be — the same defense-in-depth principle that mattered in the aviation and healthcare engagements’ incident responses, applied here to cost risk rather than safety or privacy risk.
-

16. Enterprise Architecture Artifacts

- **Capability Map:** NOC and customer care domains with AI-opportunity overlay (Section 5.4)
 - **Multi-Agent Handoff Contract Specification,** including the post-incident revision resolving the dual-service partial-outage ambiguity
 - **Cost Governance Architecture,** including the pre- and post-incident monitoring design, documented as a comparison to illustrate the specific gap identified
 - **Customer Context Service Federation Diagram,** showing per-source-system data-handling term enforcement across the three legacy CRM systems
-

17. Architecture Decision Records (ADRs)

ADR-001: NOC copilot is a single tool-equipped reasoning agent, not multi-agent; customer care is a multi-agent orchestration given its need for specialized tool access and escalation paths per domain. Status: Accepted. See Section 6.2.

ADR-002: Customer Context Service is a real-time federated view across legacy CRM systems, not a consolidated data store, preserving each source system's original data-handling terms. Status: Accepted. See Section 6.1.

ADR-003: Cost-tiered model routing, empirically validated per task class against a quality-holdout evaluation set, rather than intuition-driven model assignment. Status: Accepted. See Section 7.1.

ADR-004: Deterministic, ticket-reference-based agent-to-agent handoff contracts; no partial-confidence handoffs between specialized agents. Status: Accepted, revised post-incident to resolve the dual-service partial-outage ambiguity (Section 14.3).

ADR-005: Circuit breakers deferred from launch scope as a fast-follow item (Week 20 ARB decision), against the EAA's formal documented dissent. Status: Superseded post-incident by ADR-007.

ADR-006: Semantic caching for high-repeat query patterns (NOC alarm patterns, outage-status customer inquiries), TTL-bound and invalidated on underlying status-source updates. Status: Accepted. See Section 7.2.

ADR-007 (post-incident): Mandatory agent-to-agent circuit breakers — real-time loop-signature detection and hard handoff-cycle ceilings — required platform-wide for all multi-agent workflows, non-optional. Status: Accepted, emergency-built and deployed within three weeks of the Month 9 incident.

ADR-008 (post-incident, governance): All ARB-documented risk-acceptance decisions require a binding remediation deadline with automatic escalation on slippage. Status: Accepted. See Section 8.3.

18. AI Evaluation Strategy

- **Cost-per-interaction tracking:** continuous, per-agent and per-task-class, against explicit budget targets set at design time (Section 5.1), reviewed at every quarterly ARB session per the post-incident governance addition
 - **Cost-tier routing quality validation:** empirical quality-degradation measurement for any task routed to a cheaper model tier, against a quality-holdout set, before adopting the cheaper tier as default
 - **NOC triage accuracy:** measured against senior-engineer-validated root-cause determinations, tracked separately for pattern types well-represented in historical training data versus novel alarm signatures
 - **Customer care first-contact resolution and handoff-loop monitoring:** post-incident addition — explicit tracking of handoff-cycle counts per interaction as a leading indicator, not just a lagging cost metric
 - **Circuit-breaker efficacy testing:** adversarial test suite simulating known and hypothesized loop-inducing edge cases, run against every handoff-contract change before production deployment
-

19. Operational Runbook

- **Real-time loop-signature monitoring:** dashboard and alerting distinct from aggregate daily cost tracking, specifically designed post-incident to detect the narrow-category, fast-accumulating pattern that evaded the original monitoring
 - **Emergency circuit-breaker override procedure:** documented steps for manually disabling a specific agent-to-agent handoff path if a new loop pattern is suspected, without requiring a full platform rollback
 - **Major-event elevated-volume runbook:** distinguishes expected aggregate volume increases (e.g., during a declared outage) from per-interaction anomalies, informed directly by the Month 9 incident's core monitoring gap
 - **BPO partner incident escalation path:** defined procedure for BPO-originated sessions given their distinct identity class and support relationship
-

20. Future Roadmap

1. **IVR replacement initiative** (deferred to a separate, parallel program per Section 5.4), designed to integrate with the existing voice-channel agent orchestration once a vendor is selected.
2. **Expansion of the NOC copilot's learned-correlation model** to additional incident categories beyond the initial pilot scope, contingent on continued accuracy validation against senior-engineer judgment.
3. **Predictive churn-intervention deepening**: closer integration between the Retention agent and Vantar's existing churn-prediction model, potentially enabling more proactive (not just reactive) retention outreach — explicitly flagged as needing its own discovery phase given the different consent and customer-experience considerations of proactive versus reactive contact.
4. **Circuit-breaker pattern generalization**: Platform Architecture has proposed applying the post-incident loop-detection architecture as a standard, reusable component for any future multi-agent system Vantar builds, not just this platform — formally adopted as an enterprise architecture standard following the postmortem.